

Методи на адресиране

Ефективната разработка на софтуер за микропроцесора изисква пълно запознаване с методите на адресиране, използвани от всяка инструкция. Тук инструкцията MOV (преместване на данни) се използва за описание на методите на адресиране на данни. Инструкцията MOV прехвърля байтове или думи от данни между регистри или между регистри и памет в 8086 до 80286 и байтове, думи или двойни думи в 80386 и по-горе. При описанието на методите на адресиране на паметта на програмата, инструкциите CALL и JMP показват как се модифицира потока на програмата. Методите на адресиране на данни включват регистров, непосредствен, директен, косвен регистров, базово-индексен, в микропроцесорите 8086 до 80286. 80386 и по-горе също включват метод на мащабирано индексно адресиране на данни от паметта. Методите на адресиране на паметта от програмата включват относително, директно и косвено програмно. Обяснена е работата на стековата памет е.

Методи на адресиране на данни

Инструкцията MOV осигурява основа за обяснението на режимите на адресиране на данни. Фигура 3-1 илюстрира инструкцията MOV и дефинира посоката на потока от данни. Източникът е отдясно, а получателят е отляво, до мнемокода на операцията MOV. Мнемокодът на операцията казва на микропроцесора коя операция да изпълни. Данните тук се движат отдясно наляво. Запетаята в инструкцията винаги разделя получателя от източника. Прехвърлянето на данни от памет към памет не е разрешено, с изключение на инструкцията MOVS.

На Фигура 3-1 инструкцията MOV AX,BX прехвърля съдържанието на думата на изходния регистър (BX) в регистъра на получателя (AX). Източникът никога не се променя, но получателят се променя почти винаги. Инструкцията MOV винаги копира изходните данни в адреса на получателя. Регистърът на флаговете не се променя от повечето инструкции за трансфер на данни. Фигура 3-2 показва всички възможни варианти на режимите за адресиране на данни, използвайки MOV инструкцията. Тази илюстрация показва как се записва инструкцията при различните методи на адресиране на данни с инструкцията MOV. Тези методи на адресиране на данни се срещат във всички версии на микропроцесора Intel, с изключение на режима на мащабирано индексно адресиране, който се среща в 80386 - Pentium Pro. Методите за адресиране на данните са:

Регистрова адресация

Прехвърля копие на байт или дума от изходния регистър или място в паметта в целевия регистър или място в паметта. (Пример: инструкцията MOV CX,DX копира съдържанието на регистър DX с размер на дума в регистър CX.) В 80386 и по-горе, двойна дума може да бъде прехвърлена от изходния регистър или местоположението на паметта към регистъра на местоназначението или местоположението на паметта . (Пример: инструкцията MOV ECX,EDX копира съдържанието на регистър EDX с размер на двойна дума в регистър ECX.)

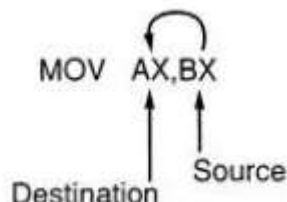
Непосредствена адресация

Прехвърля непосредствения източник на байт или дума от данни в регистъра на местоназначението или място в паметта. (Пример: инструкцията MOV AL,22H копира 22H с размер на байт в регистър AL.) В 80386 и по-горе, двойна дума от непосредствени данни може да бъде прехвърлена в регистър или място в паметта. (Пример: инструкцията MOV EBX,12345678H копира 12345678H с размер на двойна дума в 32-битовия EBX регистър.)

Директна адресация

Премества байт или дума между адрес на паметта и регистър. Наборът от инструкции не поддържа прехвърляне от памет към памет, с изключение на инструкцията MOVS. Пример: инструкцията MOV CX,LIST копира съдържанието на паметта LIST с размер на дума в регистър CX.) В 80386 и по-горе може да се адресира и местоположение на паметта с размер на двойна дума. (Пример: инструкцията MOV ES,LIST копира 32-битово число, съхранено в четири последователни байта памет, от местоположение LIST в регистър ESI.)

FIGURE 3-1 The MOV instruction showing the source, destination, and direction of data flow



Регистрова косвена адресация

Прехвърля байт или дума между регистър и място в паметта, адресирано от индексен или основен регистър. Индексните и базовите регистри са BP, BX, DI, и SI. (Пример: инструкцията MOV AX,[BX] копира данните с размер на дума от адреса на изместване на сегмента от данни, индексирани от BX, в регистър AX.) В 80386 и по-горе, байт, дума или двойна дума се прехвърлят между регистър и място в паметта, адресирано от всеки регистър: EAX, EBX, ECX, EDX, EBP, EDI или ESI. (Пример: инструкцията MOV AL,[ECX] зарежда AL от адреса на изместване на сегмента от данни, избран от съдържанието на ECX.)

Базово-индексна адресация

Прехвърля байт или дума между регистър и мястото на паметта, адресирано от основен регистър (BP или BX) плюс индексен регистър (DI или SI). (Пример: инструкцията MOV [BX+DI],CL копира съдържанието на регистър CL с размер на байта в местоположението на паметта на сегмента от данни, адресирано от BX плюс DI.) В 80386 и по-горе, всеки регистър EAX, EBX, ECX, EDX, EBP, EDI или ESI могат да бъдат комбинирани за генериране на адреса на паметта. (Пример: инструкцията MOV [EAX+EBX],CL копира съдържанието на регистъра CL с размер на байтове в местоположението на паметта на сегмента от данни, адресирано от EAX плюс EBX.)

Косвена регистрова адресация

Премества байт или дума между регистър и мястото на паметта, адресирано от индексен или основен регистър плюс изместване. (Пример: MOV AX,[BX+4] или MOV AX,ARRAY[BX]. Първата инструкция зарежда AX от адреса на сегмента от данни, образуван от BX плюс 4. Втората инструкция зарежда AX от местоположението на паметта на сегмента от данни в ARRAY плюс съдържанието на BX.) 80386 и по-нови използват всеки регистър за адресиране на паметта. (Пример: MOV AX,[ECX+4] или MOV AX,ARRAY[EBX]. Първата инструкция зарежда AX от адреса на сегмента от данни, образуван от ECX плюс 4. Втората инструкция зарежда AX от мястото на паметта на сегмента от данни ARRAY плюс съдържанието на EBX.)

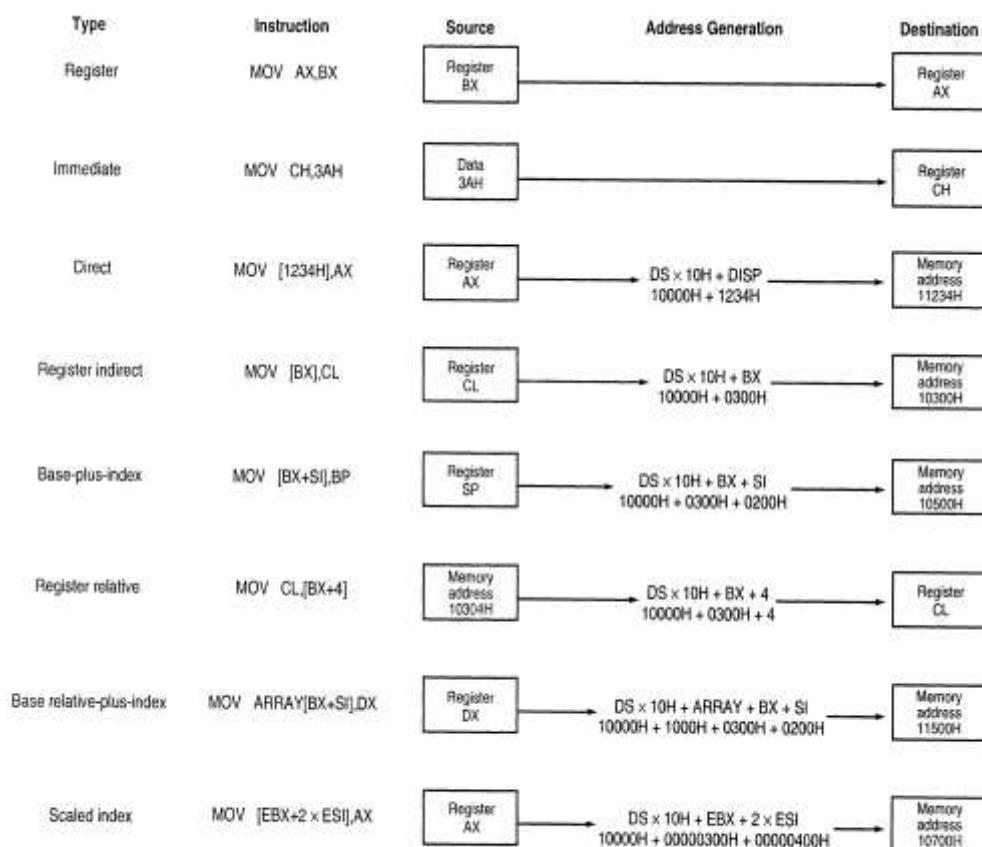
Машабирана индексна адресация

Предлага се в 80386 - Pentium Pro. Вторият регистър на двойка регистри се умножава с мащабен коефициент 2X, 4X, или 8X за получаване на адреса на паметта на операнда. (Пример: инструкция MOV EDX,[EAX+4*EBX] зарежда EDX от местоположението на паметта на сегмента от данни, адресирано от EAX плюс 4 пъти EBX.) Машабирането позволява достъп до дума (2X), двойна дума (4X) или четворна дума (8X) данни от масива памет. Имайте предвид, че коефициент на машабиране на IX също съществува, но той обикновено се подразбира и не се появява в инструкцията. MOV AL,[EBX+ECX] е пример, при който коефициентът на машабиране е единица.

Инструкцията може да бъде записана като `MOV AL,[EBX+1*ECX]`. Друг пример е инструкцията `MOV AL,[2*EBX]`, която използва само един мащабиран регистър за адресиране на паметта.

Косвена базова-индексна адресация

Прехвърля байт или дума между регистър и мястото на паметта, адресирано от основен и индексен регистър плюс изместване. (Пример: `MOV AX,ARRAY[BX+DI]` или `MOV AX,[BX+DI+4]`. И двете инструкции зареждат AX от място в паметта на сегмента от данни. Първата инструкция използва адрес, образуван чрез добавяне на ARRAY, BX, и DI; вторият, чрез добавяне на BX, DI и 4.) (Пример 80386 и по-горе: `MOV EAX,ARRAY[EBX+ECX]` зарежда EAX от местоположението на паметта на сегмента от данни, достъпно от сумата от ARRAY, EBX, и ECX.)



Notes: EBX = 00000300H, ESI = 00000200H, ARRAY = 1000H, and DS = 1000H

FIGURE 3-2 8086-Pentium-Pro data-addressing modes

Регистрова адресация

Адресирането чрез регистъра е най-често срещаната форма на адресиране на данни. Микропроцесорът съдържа следните 8-битови регистри, използвани с адресиране в регистъра: AH, AL, BH, BL, CH, CL, DH, и DL. Присъстват също и следните 16-битови регистри: AX, BX, CX, DX, SP, BP, SI и DI. В 80386 и по-горе разширените 32-битови регистри са EAX, EBX, ECX, EDX, ESP, EBP, EDI, и ESI. С регистровото адресиране, някои инструкции MOV и инструкциите PUSH и POP също използват 16-битовите сегментни регистри (CS, ES, DS, SS, FS и GS). За инструкциите е важно да се използват регистри с еднакъв размер. Никога не смесвайте 8-битов регистър с 16-битов регистър, 8-битов регистър с 32-битов регистър или 16-битов регистър с 32-битов регистър, тъй като това не е позволено от микропроцесора и води до грешка при сглобяване. Това е вярно дори когато инструкцията `MOV AX,AL` или `MOV EAX,AL` може да изглежда да

има смисъл. Разбира се, инструкциите MOV AX.AL или MOV EAX.AL не са разрешени, тъй като тези регистри са с различни размери. Имайте предвид, че няколко инструкции, като SHL DX.CL, правят изключения от това правило, както е посочено в следващите глави. Също така е важно да се отбележи, че никоя MOV инструкция не засяга флаговете.

Таблица 3-1 показва много варианти на инструкции за преместване на регистър. Невъзможно е да се покажат всички възможни комбинации. Само 8-битовото подмножество на MOV инструкцията има 64 различни варианта. Инструкцията MOV от сегмент към сегмент е практически единственият тип инструкция MOV на регистър, която не е разрешена. Също така регистърът на кодовия сегмент не може директно да бъде променен от MOV инструкция.

TABLE 3-1 Examples of the register-addressed instructions

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV AL,BL	8-bits	Copies BL into AL
MOV CH,CL	8-bits	Copies CL into CH
MOV AX,CX	16-bits	Copies CX into AX
MOV SP,BP	16-bits	Copies BP into SP
MOV DS,AX	16-bits	Copies AX into DS
MOV SI,DI	16-bits	Copies DI into SI
MOV BX,ES	16-bits	Copies ES into BX
MOV ECX,EBX	32-bits	Copies EBX into ECX
MOV ESP,EDX	32-bits	Copies EDX into ESP
MOV ES,DS	—	Not allowed (segment-to-segment)
MOV BL,DX	—	Not allowed (mixed sizes)
MOV CS,AX	—	Not allowed (the code segment register may not be the destination register)

Непосредствено адресиране

Друг режим на адресиране на данни е непосредственото адресиране. Терминът означава, че данните непосредствено следват шестнадесетичния опкод в паметта. Също така непосредствените данни са постоянни данни, докато данните, прехвърлени от регистър, са променливи данни. Непосредственото адресиране оперира с байт или дума от данни. В микропроцесорите 80386 - Pentium Pro непосредственото адресиране също оперира върху данни с дължина двойна дума. Инструкцията MOV прехвърля копие на непосредствените данни в регистър или място в паметта. Фигура 3-4 показва работата на инструкция MOV EAX,13456H. Тази инструкция копира числото 13456H от инструкцията, разположена в паметта непосредствено след шестнадесетичния опкод, в регистъра EAX. Както при MOV инструкцията, илюстрирана на Фигура 3-3, данните на източника се записват върху данните на получателя.

FIGURE 3-3 The effect of executing the MOV BX, CX instruction at the point just before the BX register changes. Note that only the rightmost 16-bits of register EBX change.

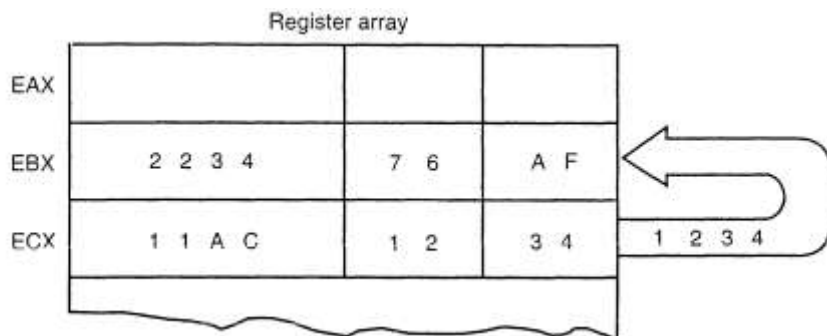
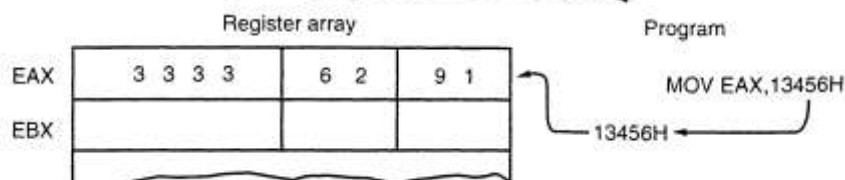


FIGURE 3-4 The operation of the MOV EAX,3456H instruction. This instruction copies the immediate data (13456H) into EAX.



Методи за адресиране на програмата в паметта

Методите за адресиране на програмата в паметта, използвани с JMP и CALL инструкции, използват 3 форми: директна, относителна и непряка. Ще покажем примери с JMP инструкция.

Директна/пряка/ адресация.

Прякото адресиране на паметта е същото като използваното при стари микропроцесори при инструкции за преход и извикване на подпрограма. То се ползва по-рядко от методите на косвено и относително адресиране.

Пример 3-2 показва различни инструкции с непосредствена адресация в кратка програма, която зарежда 0000H в 16-битовите регистри AX, BX и CX. Това е последвано от инструкции, които използват регистрово адресиране за копиране на съдържанието на AX в регистри SI, DI и BP. Инструкцията .MODEL TINY кара асемблера да разположи програмата в един сегмент. Инструкцията или директивата .CODE указва началото на кодовия сегмент; операторът .STARTUP показва началната инструкция в програмата; и операторът .EXIT кара програмата да излезе в DOS. Инструкцията END показва края на програмния файл. Тази програма може да бъде асемблирана с MASM.

EXAMPLE 3-2

```

0000          .MODEL TINY          ;choose single segment model
              .CODE                ;indicate start of code segment

              .STARTUP             ;indicate start of program

0100  B8 0000    MOV    AX,0        ;place 0000H into AX
0103  BB 0000    MOV    BX,0000H    ;place 0000H into BX
0106  B9 0000    MOV    CX,0        ;place 0000H into CX

0109  8B F0      MOV    SI,AX       ;copy AX into SI
010B  8B F8      MOV    DI,AX       ;copy AX into DI
010D  8B E8      MOV    BP,AX       ;copy AX into BP

              .EXIT                ;exit to DOS
              END                  ;end of file

```

Всеки израз в програмата се състои от четири части/ полета, както е показано в Пример 3-3. Най-лявото поле се нарича етикет и се използва за съхраняване на символно име за мястото на паметта, което представлява. Всички етикети започват с буква или с някой от специалните символи: @, \$, _ или ?. Етикетът може да бъде с произволна дължина

от 1 до 35 знака. Етикетът задава името на местоположението на паметта/адреса/ за съхранение на данни и за други цели, които са обяснени в текста. Следващото поле се нарича поле за код на операцията; той съхранява мнемокода на инструкцията или кода на операцията. Инструкцията MOV е пример за операционен код. Вдясно от полето на кода на операцията е полето на операнда, което съдържа информация, използвана от кода на операцията. Например, инструкцията MOV AL,BL има мнемокод MOV и операнди AL и BL. Някои инструкции съдържат между нула и три операнда. Последното поле е полето за коментар. То съдържа коментар за инструкцията или група инструкции. Коментарът винаги започва с точка и запетая (;).

EXAMPLE 3-3

LABEL	OPCODE	OPERAND	COMMENT
DATA1	DB	23H	;define DATA1 as a byte of 23H
DATA2	DW	1000H	;define DATA2 as a word of 1000H
START:	MOV	AL,BL	;copy BL into AL
	MOV	BH,AL	;copy AL into BH
	MOV	CX,200	;copy 200 decimal into CX

Когато (програмата е транслирана, файлът с резултата от трансляцията (.LST) изглежда като Пример 3-2. Шестнадесетичното число в най-ляво е изместването на адреса на инструкцията или данните. Той се генерира от асемблера. Числото или числата вдясно от отместването са машинно кодирани инструкции или данни, които също се генерират от асемблера. Например, ако инструкцията MOV AX,0 се появи във файл и се транслира, той се появява на място в паметта 0100 в Пример 3-2. Нейният машинен код е B8 0000. Когато програмата е написана, в редактора е въведена само инструкцията MOV AX,0; асемблерът генерира машинния код и адреса му и съхранява програмата във файл, завършващ с разширението .LST.

Директно адресиране на данни

Повечето инструкции могат да използват директно адресиране на данни. Има две основни форми на директно адресиране на данни: (1) директно адресиране, което се прилага за MOV между място в паметта и AL, AX, или EAX, и (2) адресиране с изместване, което се прилага за почти всяка инструкция в набора от инструкции. И в двата случая адресът се формира чрез добавяне на изместване към адреса на сегмента от данни по подразбиране или към адрес на алтернативен сегмент.

Директно адресиране.

Директното адресиране с инструкцията MOV прехвърля данни между място на паметта, разположено в сегмента от данни, и регистъра AL (8-битов), AX (16-битов) или EAX (32-битов). MOV инструкция, използваща този тип адресиране, обикновено е 3-байтова инструкция. (В 80386 и по-нови може да има префикс за размер на регистъра преди инструкцията, което кара кода да надвиши три байта.)

Адресиране с изместване.

Адресирането с изместване е почти идентично с директното адресиране, с изключение на това, че машинният код на инструкцията е дълга четири байта вместо три. В 80386 - Pentium Pro тази инструкция може да бъде дълга до седем байта, ако е указан 32-битов регистър и 32-битово изместване. Този тип директно адресиране на данни е много по-гъвкав и повечето инструкции го използват.

Таблица 3-4 изброява някои MOV инструкции, използващи формата на изместване при директно адресиране. Не всички варианти са показани, защото има много MOV инструкции от този тип. Сегментните регистри могат да се съхраняват или зареждат от паметта.

TABLE 3-4 Examples of direct data addressing using a displacement

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV CH,DOG	8-bits	Copies the byte contents of data segment memory location DOG into CH
MOV CH,[1000H]	8-bits	Copies the byte contents of data segment offset address 1000H into CH
MOV ES,DATA6	16-bits	Copies the word contents of data segment memory location DATA6 into ES
MOV DATA7,BP	16-bits	Copies BP into data segment memory location DATA7
MOV NUMBER,SP	16-bits	Copies SP into data segment memory location NUMBER
MOV DATA1,EAX	32-bits	Copies EAX into data segment memory location DATA1
MOV EDI,SUM1	32-bits	Copies the doubleword contents of data segment memory location SUM1 into EDI

Пример 3-5 показва кратка програма за адресирана на информация в сегмента от данни. Сегментът от данни започва с директивата `.DATA`. Размерът на модела се настройва от атрибут в операндното поле `TINY`, както в пример 3-3, или `SMALL`. Моделът `SMALL` позволява един сегмент от данни и един сегмент от код. Моделът `SMALL` често се използва винаги, когато са необходими данни от паметта за програмата. `SMALL` модел програма се транслира като изпълнима програма (`.EXE`). В този пример място в паметта и в сегмента от данни се заделя, използвайки директивите `DB` и `DW`. Тук операторът `.STARTUP` указва началото на кода и зарежда регистъра на сегмента от данни с адреса на сегмента за данни. Ако тази програма е транслирана и изпълнена с `CodeView`, може да се види как се изпълняват инструкциите и как променят регистри и клетки от паметта.

EXAMPLE 3-5

```

0000                                .MODEL SMALL      ;select SMALL model
                                .DATA                ;indicate start of DATA segment

0000 10      DATA1  DB    10H      ;place 10H in DATA1
0001 00      DATA2  DB     0      ;place 0 in DATA2
0002 0000    DATA3  DW     0      ;place 0 in DATA3
0004 AAAA    DATA4  DW   0AAAAH   ;place AAAAH in DATA4

0000                                .CODE             ;indicate start of CODE segment
                                .STARTUP            ;indicate start of program

0017 A0 0000 R      MOV     AL,DATA1  ;copy DATA1 to AL
001A 8A 26 0001 R   MOV     AH,DATA2  ;copy DATA2 to AH
001E A3 0002 R      MOV     DATA3,AX  ;save AX at DATA3
0021 8B 1E 0004 R   MOV     BX,DATA4  ;load BX with DATA4

                                .EXIT              ;exit to DOS
                                END              ;end file

```

Косвена регистрова адресация

Индиректното адресиране с регистър позволява адресирането на данни във всяко място в паметта чрез отместване, съхранявано в някой от следните регистри: BP, BX, DI и SI. Ако регистър BX съдържа 1000H и се изпълни инструкцията MOV AX,[BX], съдържанието на думата от адрес на изместване 1000H от началото на сегмента за данни се копира в регистър AX. Ако микропроцесорът работи в реален режим и DS = 0100H, тази инструкция адресира дума, съхранена в паметта байтове 2000H и 2001H и я прехвърля в регистър AX (виж Фигура 3-6).

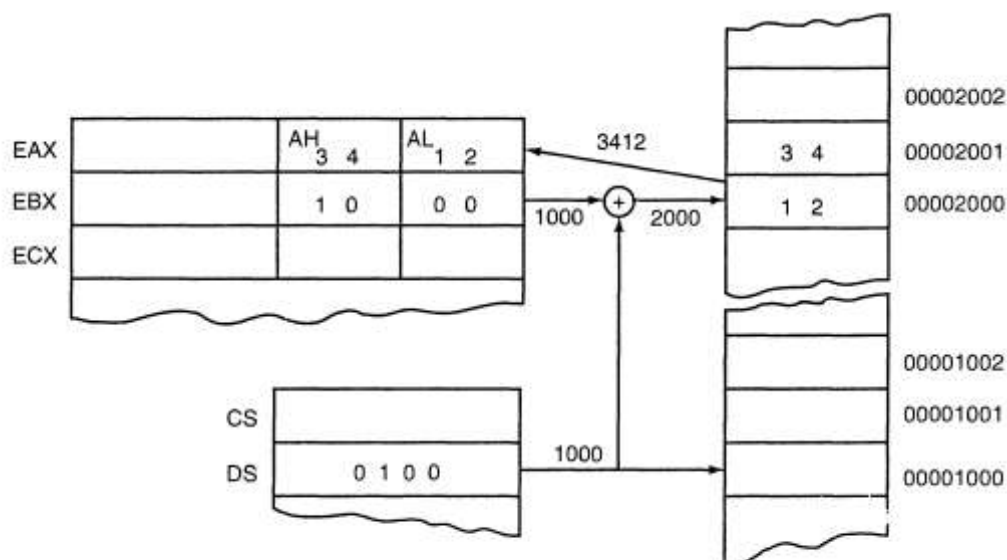


FIGURE 3-6 The operation of the MOV AX,[BX] instruction when BX = 1000H and DS = 0100H. Note that this instruction is shown after the contents of memory are transferred to AX.

Съдържанието на клетка с адрес 2000H се премества в AL, а съдържанието на 2001H се премества в AH. [] символите означават непряко адресиране на езика на асемблирането. В допълнение към използването на регистрите BP, BX, DI и SI за индиректно адресиране на паметта, 80386 и по-високи позволяват непряко адресиране с регистър с всеки разширен регистър с изключение на ESP. Някои типични инструкции с помощта на непряко адресиране са показани в Таблица 3-5.

TABLE 3-5 Example of register indirect addressing

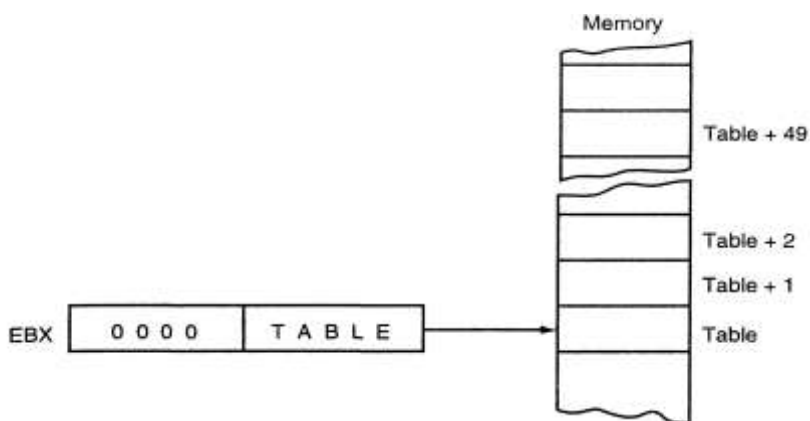
Assembly Language	Size	Operation
MOV CX,[BX]	16-bits	Copies the word contents of the data segment memory location address by BX into CX
MOV [BP],DL*	8-bits	Copies DL into the stack segment memory location addressed by BP
MOV [DI],BH	8-bits	Copies BH into the data segment memory location addressed by DI
MOV [DI],[BX]	—	Memory-to-memory moves are not allowed except with string instructions
MOV AL,[EDX]	8-bits	Copies the byte contents of the data segment memory location addressed by EDX into AL
MOV ECX,[EBX]	32-bits	Copies the doubleword contents of the data segment memory location addressed by EBX into ECX

*Note: Data addressed by BP or EBP is by default located in the stack segment, while all other indirect addressing modes use the data segment by default.

Сегментът от данни се използва по подразбиране с непряко адресиране чрез регистър или всеки друг режим на адресиране, който използва BX, DI или SI за адресиране на паметта. Ако регистърът BP адресира паметта, сегментът на стека се използва по подразбиране. Те се считат за сегменти по подразбиране за тези четири регистри. За 80386 и по-горе, EBP адресира паметта в сегмента на слаби по подразбиране: EAX

В някои случаи непрякото адресиране изисква специфициране на размера на данните със специалната асемблерна директива BYTE PTR, WORD PTR или DWORD PTR. Тези директиви показват размера на данните от паметта, адресирани от указателя на паметта (PTR). Например, инструкцията MOV AL,[DI] е инструкция за преместване на данни с размер байт, но инструкцията MOV [DI],10H е двусмислена, защото не е ясно дали адресира байт, дума или клетка с размер на двойна дума. Асемблерът не може да определи размера на 10H. Инструкцията MOV BYTE PTR [DI],10H ясно обозначава данните, адресирани от DI в паметта с размер байт. По същия начин инструкцията MOV DWORD PTR [DI],10H ясно идентифицира данните в паметта с размер двойна дума. Директивите BYTE PTR, WORD PTR и DWORD PTR се използват само с инструкции, които адресират място в паметта чрез указател или индексен регистър с непосредствени данни и за няколко други инструкции. Непрякото адресиране често позволява на програмата да се позовава на таблични данни, намиращи се в системната памет. Нека трябва да се създаде таблица с информация, която съдържа 50 отчета, взети от паметта с адрес 0000:046C. Клетка 0000:046C съдържа брояч, който се поддържа от часовника за реално време на персоналния компютър. Фигура 3-7 показва таблицата и регистъра BX, използвани за последователно адресиране на всяко място в таблицата. За да се изпълни задачата, началния адрес на таблицата се зарежда в регистър BX с инструкция MOV използваща непосредствен метод на адресация. След като е зададен началния адрес на таблицата, се ползва индиректно адресиране чрез регистър, за да се съхранят последователно 50-те отчета.

FIGURE 3-7 An array (TABLE) containing 50 bytes that are indirectly addressed through register BX



Базово-индексно адресиране

Базово-индексното адресиране непряко адресира данните от паметта. В 8086 - 80286. този тип адресиране използва един основен регистър (BP или BX) и един индексен регистър (DI или SI) за индиректно адресиране на паметта. Базовият регистър често сочи началния адрес на масива с памет, докато индексният регистър съдържа относителната позиция на елемент в масива. Всеки път, когато BP адресира данни от паметта, както сегментният регистър на стека, така и BP генерират ефективния адрес. В 80386 и по-горе този тип адресиране позволява комбинацията от всеки два 32-битови разширени регистъра с изключение на ESP. Например инструкцията MOV DL,[EAX+EBX] е пример за използване на EAX (като база) плюс EBX (като индекс).

Ако се използва регистърът ЕВР, данните се намират в сегмента на стека, вместо в сегмента от данни.

Фигура 3-8 показва как данните се адресират от инструкцията MOV DX,[BX+DI], когато микропроцесорът работи в реален режим. В този пример BX = 1000H, DI = 0010H и DS = 0100H, което указва адрес на паметта 02010H. Тази инструкция прехвърля копие на думата от местоположение 02010H в DX регистъра. Таблица 3-6 изброява някои инструкции, използвани при базово-индексна адресация.

TABLE 3-6 Examples of base-plus-index addressing

Assembly Language	Size	Operation
MOV CX,[BX+DI]	16-bits	Copies the word contents of the data segment memory location address by BX plus DI into CX
MOV CH,[BP+SI]	8-bits	Copies the byte contents of the stack segment memory location addressed by BP plus SI into CH
MOV [BX+SI],SP	16-bits	Copies SP into the data segment memory location addresses by BX plus SI
MOV [BP+DI],AH	8-bits	Copies AH into the stack segment memory location addressed by BP plus DI
MOV CL,[EDX+EDI]	8-bits	Copies the byte contents of the data segment memory location addressed by EDX plus EDI into CL
MOV [EAX+EBX],ECX	32-bits	Copies ECX into the data segment memory location addressed by EAX plus EBX

Асемблерът на Intel указва този режим на адресиране като [BX][DI] вместо [BX+DI].

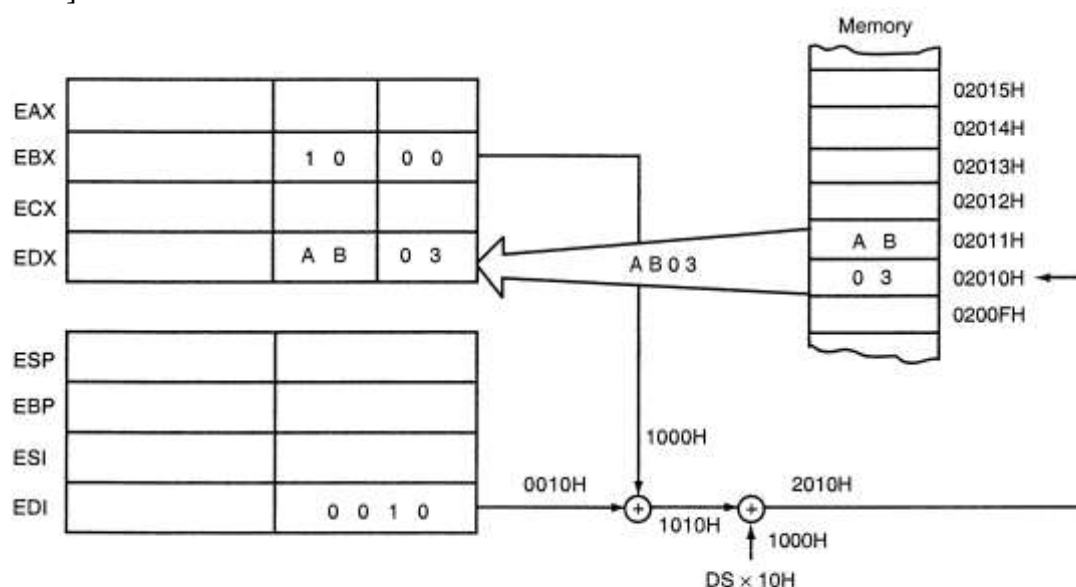


FIGURE 3-8 An example showing how the base-plus-index addressing mode functions for the MOV DX,[BX+DI] instruction. Notice that memory address 02010H is accessed because DS = 0100H, BX = 100H, and DI = 0010H.

Базово-индексно адресиране се използва основно при адресирането на елементи в масив в паметта. Нека елементите на масива ARRAY се намират в сегмента от данни на паметта. За да бъде масива достъпен, се зарежда регистъра BX (базата) с началния адрес на масива и индексния регистър DI с номера на елемента, до който ще се прави

достъп. Фигура 3-9 показва използването на BX и DI за достъп до елемент в масив от данни.

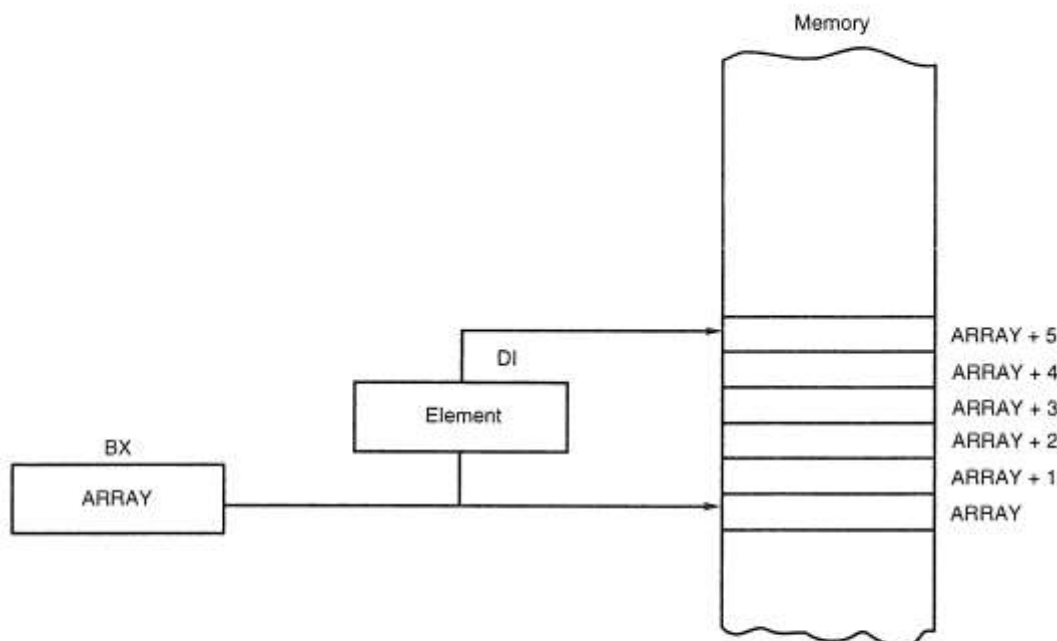


FIGURE 3-9 An example of the base-plus-index addressing mode. Here an element (DI) of an ARRAY (BX) is addressed.

Относителна регистрова адресация

При относителната регистрова адресация данните в сегмент от паметта се адресират чрез добавяне на изместване към съдържанието на основен или индексен регистър (BP, BX, DI или SI). Фигура 3-10 показва действието на инструкцията MOV AX,[BX+1000H]. В този пример BX = 0100H и DS = 0200H. Генерираният адрес е сумата от DS x 10H, BX и отместването - 1000H или 03100H. BX, DI, или SI адресира сегмента от данни, а BP адресира сегмента на стека. В 80386 и по-високи, отместването може да бъде 32-битово число, а регистърът може да бъде всеки 32-битов регистър с изключение на регистъра ESP. Размерът на сегмента в реален режим е 64K байта. Таблица 3-7 изброява няколко инструкции, които използват относително адресиране на регистри.

TABLE 3-7 Examples of register relative addressing

Assembly Language	Size	Operation
MOV AX,[DI+100H]	16-bits	Copies the word contents of the data segment memory location addressed by DI plus 100H into AX
MOV ARRAY[SI],BL	8-bits	Copies BL into the data segment memory location addressed by ARRAY plus SI
MOV LIST[SI+2],CL	8-bits	Copies CL into the data segment memory location addressed by sum of LIST, SI, and 2
MOV DI,SET_IT[BX]	16-bits	Copies the word contents of the data segment memory location addressed by the sum of SET_IT and BX into DI
MOV DI,[EAX+10H]	16-bits	Copies the word contents of the data segment memory location addressed by the sum of EAX and 10H into DI
MOV ARRAY[EBX],EAX	32-bits	Moves EAX into the data segment memory location addressed by the sum of ARRAY and EBX

Отместването може да бъде число, добавено към регистъра в скобите [], както в `MOV AL, [DI+2]`, или може да бъде изместване, извадено от регистъра, както в `MOV AL, [SI-1]`. Изместването също може да бъде адрес на изместване, добавен към етикета пред [], както в `MOV AL, DATA[DI]`. И двете форми на отместване също могат да се появят едновременно, както в `MOV AL, DATA[DI+3]`. Във всички случаи и двете форми на изместване се добавят към базата или базата и индексния регистър в скобите []. В микропроцесорите 8086-80286 стойността на изместването е ограничена до 16-битово число със знак със стойност между +32,767 (7FFFH) и -32,768 (8000H); в 80386 и по-високи е разрешено 32-битово изместване със стойност между +2.147.483.647 (7FFF FFFFH) и -2.147,483,648 (8000 0000H).

Адресиране на данните в масив ARRAY с относителна регистрова адресация

Възможно е да се адресират данните от масива с относително адресиране чрез регистър, напр. с базово-индексно адресиране. На Фигура 3-11 относителното адресиране на регистъра е илюстрирано със същия пример като при базово-индексно адресиране. Изместването ARRAY добавя към индексния регистър DI, за да генерира адреса в паметта на елемента на масива.

3-1 DATA-ADDRESSING MODES

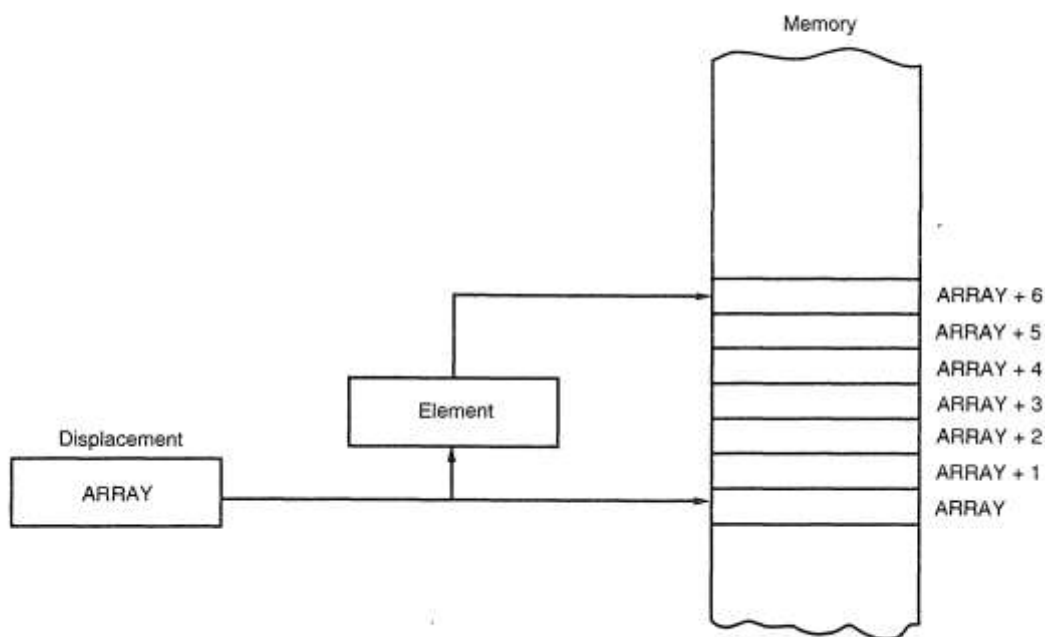


FIGURE 3-11 Register relative addressing used to address an element of ARRAY. The displacement addresses the start of ARRAY, and DI accesses an element.

Относителна базово- индексна адресация

Относителното базово-индексно адресиране е подобно на режима на базово-индексното адресиране, но добавя изместване освен използването на базата и индексен регистър за формиране на адреса на паметта. С този режим на адресиране често се адресира двумерен масив от данни в паметта.

Адресиране на данни чрез относителна базово-индексна адресация.

Относителната базово-индексна адресация е най-рядко използвания метод на адресиране. Фигура 3-12 показва как се отнасят към данните, ако инструкцията, изпълнявана от микропроцесора, е `MOV AX,[BX+SI+100H]`.

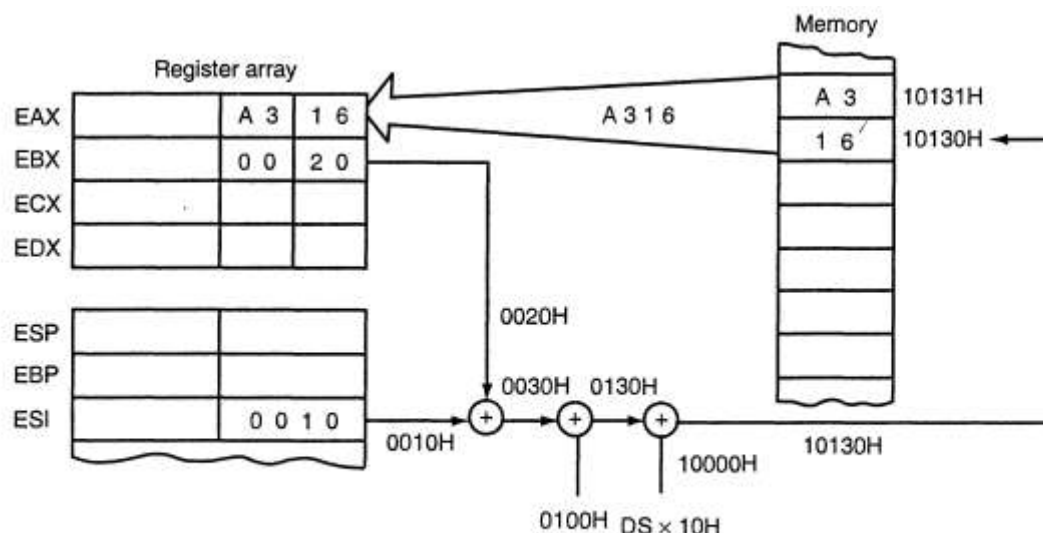


FIGURE 3-12 An example of base relative-plus-index addressing using a `MOV AX,[BX+SI+100H]` instruction. Note: `DS = 1000H`.

Изместването 100H се добавя към BX и SI, за да се получи отместването в сегмента от данни. Регистри BX = 0020H, SI = 0010H и DS = 1000H, така че ефективният адрес за тази инструкция е 10130H – сумата от тези регистри плюс изместването 100H. Този режим на адресиране е твърде сложен за често използване в програма. Някои типични инструкции, използващи базово относително-индексно адресиране са показани в Таблица 3-8. При 80386 и по-високи, ефективният адрес се генерира от сумата от два 32-битови регистъра плюс 32-битово отместване.

TABLE 3-8 Example base relative-plus-index instructions

Assembly Language	Size	Operation
<code>MOV DH,[BX+DI+20H]</code>	8-bits	Copies the byte contents of the data segment memory location addressed by the sum of BX, DI, and 20H into DH
<code>MOV AX,FILE[BX+DI]</code>	16-bits	Copies the word contents of the data segment memory location addressed by the sum of FILE, BX, and DI into AX
<code>MOV LIST[BP+DI],CL</code>	8-bits	Copies CL into the stack segment memory location addressed by the sum of LIST, BP, and DI
<code>MOV LIST[BP+SI+4],DH</code>	8-bits	Copies DH into the stack segment memory location addressed by the sum of LIST, BP, SI, and 4
<code>MOV EAX,FILE[EBX+ECX+2]</code>	32-bits	Copies the doubleword contents of the data segment memory location addressed by the sum of FILE, EBX, ECX, and 2 into EAX

Машабирано индексно адресиране

Машабираното индексно адресиране на данни е уникално за 80386 - Pentium Pro микропроцесори. То използва два 32-битови регистъра (базов регистър и индексен регистър) за достъп до паметта. Вторият регистър (индекс) се умножава по коефициент на машабиране. Коефициентът на машабиране може да бъде 1X, 2X, 4X или 8X. Коефициент на машабиране 1X се подразбира и не е необходимо да се включва в инструкциите на асемблерния език (`MOV AL, [EBX+ECX]`). Коефициент на машабиране 2X се използва за адресиране на масиви памет с размер на думи; машабиращ фактор 4X се използва с масиви памет с размер на двойна дума; и коефициент на машабиране 8X се използва с масиви памет с размер четири думи. Примерна инструкция е `MOV AX, [EDI+2*ECX]`. Тази инструкция използва

машабиращ коефициент 2X, който умножава съдържанието на ECX по 2, преди да го добави към регистъра на EDI, за да образува адреса на паметта. Ако ECX съдържа 00000000H, се адресира елемент от памет с размер на дума 0; ако ECX съдържа 00000001H, се адресира елемент от памет с размер на дума 1 и т.н. Това машабира индекса (ECX) с коефициент 2 за масив с памет с размер на думи. Вижте Таблица 3-9 за някои примери за адресиране с машабиран индекс. Както се очаква, има изключително голям брой комбинации от адресирани регистри с машабиран индекс.

TABLE 3-9 Examples of scaled-index addressing

Пример 3-10 показва последователност от инструкции, която използва адресиране с мащабиран индекс за достъп до масив от данни с размер на дума, наречен LIST. Адресът на отместването на LIST се зарежда в регистър EBX с инструкцията MOV EBX,OFFSET LIST. След като EBX адресира масив LIST, елементите (разположени в ECX) от 2, 4 и 7 от този масив в цялата дума се добавят с помощта на коефициент на мащабиране 2 за достъп до елементите. Тази програма съхранява 2 в елемент 2 в елементи 4 и 7. Също така директивата .386 избира микропроцесор 80386. Тази директива трябва да следва оператора .MODEL, за да може асемблерът да обработва 80386 инструкции за DOS. Ако се използва 80486, директивата .486 се появява след израза .MODEL; ако се използва Pentium или Pentium Pro, директивата .586 се появява след израза .MODEL. Ако директивата за избор на микропроцесор се появи преди оператора .MODEL, микропроцесорът изпълнява инструкции в 32-битов режим, който не е съвместим с DOS.

Address	Disassembly	Comment
	.MODEL SMALL	;select SMALL model
	.386	;use the 80386
0000	.DATA	;start of DATA segment
0000	0000 0001 0002 LIST DW 0,1,2,3,4	;define array list
	0003 0004	
000A	0005 0006 0007 DW 5,6,7,8,9	
	0008 0009	
0000	.CODE	;start of CODE segment
	.STARTUP	;start of program
0010	66 BB 00000000 R MOV EBX,OFFSET LIST	;address array LIST
0016	66 B9 00000002 MOV ECX,2	;get element 2
001C	67& 8B 04 4B MOV AX,[EBX+2*ECX]	
0020	66 B9 00000004 MOV ECX,4	;store in element 4
0026	67& 89 04 4B MOV [EBX+2*ECX],AX	
002A	66 B9 00000007 MOV ECX,7	;store in element 7
0030	67& 89 04 4B MOV [EBX+2*ECX],AX	
	.EXIT	;exit to DOS
	END	;end of file

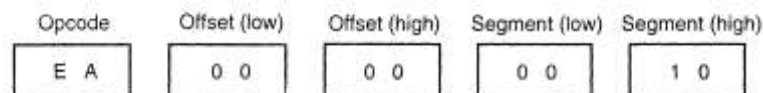
Методи за адресиране на паметта за програмата

Режимите на адресиране на паметта за програмата, използвани с инструкциите JMP и CALL, използват три различни форми: директна, относителна и индиректна. Този раздел илюстрира тези три форми за адресиране, като се използва JMP инструкцията.

Директно адресиране на паметта за програмата

Директното адресиране на програмната памет е това, което много ранни микропроцесори са използвали за всички скокове и извиквания. Директното адресиране на програмната памет се използва и в езици от високо ниво, като инструкциите GOTO и GOSUB на езика BASIC. Микропроцесорът използва тази форма на адресиране, но не толкова често, колкото се използва относително и непряко адресиране на програмната памет. Инструкциите за директно адресиране на програмната памет съхраняват адреса с операционния код. Например, ако програма прескочи до адрес в паметта 10000H за следващата инструкция, адресът 10000H се съхранява след операционния код в паметта. Фигура 3-14 показва директната междусегментна JMP инструкция и четирите байта, необходими за съхраняване на адрес 10000H. Тази JMP инструкция зарежда CS с 1000H и IP с 0000H, за да премине към адрес в паметта 10000H за следващата инструкция. (Междусегментният скок е преход до всяко място в цялата система памет.) Директният скок често се нарича далечен скок, тъй като може да прескочи до всяко място в паметта за следващата инструкция. В реалния режим, далечен скок има достъп до всяко място в рамките на първия 1 М байт памет чрез промяна на CS и IP. При работа в защитен режим далечният скок осъществява достъп до нов дескриптор на кодов сегмент от таблицата с дескриптори, което му позволява да прескача до всяко място в паметта в целия диапазон на адресите от 4G байт в 80386-Pentium Pro микропроцесори.

FIGURE 3-14 The 5-byte machine language version of a JMP [10000H] instruction

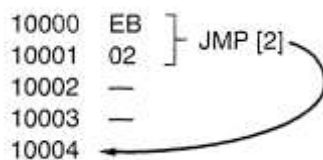


Единствената друга инструкция, която използва директно програмно адресиране, е междусегментната инструкция CALL за извикване на подпрограма. Обикновено адреса на паметта, който се извиква или към който се прескача се отбелязва с етикет, вместо действителния цифров адрес. Когато се използва етикет с инструкцията CALL или JMP, асемблерът избира най-добрата форма на програмно адресиране.

Относително адресиране на паметта на програмата

Относителното адресиране на програмната памет се ползва за фамилията микропроцесори на Intel. Терминът относителен означава „относно програмния брояч (IP)“. Ако JMP инструкция пропусне следващите два байта памет, към програмния брояч се добавя 2. Това дава адреса на следващата програмна инструкция. Пример за относителната JMP инструкция е показана на Фигура 3-15. JMP инструкцията е еднобайтова инструкция с еднобайтово или двубайтово изместване, което се добавя към IP. Еднобайтово изместване се използва при къси скокове, а двубайтово изместване се използва при близки скокове и повиквания. И двата вида се считат за вътрешносегментни скокове (скок в рамките на текущия кодов сегмент). В 80386 и по-високи, изместването може да бъде и 32-битова стойност, което позволява на тези микропроцесори да използват относително адресиране до всяко място в рамките на техните 4G байт кодови сегменти.

FIGURE 3-15 A JMP [2] instruction. This instruction skips over the two bytes of memory that follow the JMP instruction.



При относителната адресация за инструкции JMP и CALL кодът съдържа преместване, което е 8-битово или 16-битово цяло число със знак, позволяващ препращане напред или назад по адресите на паметта. (80386 и по-високи може да имат 8-битово или 32-битово изместване.) Всички асемблери автоматично изчисляват изместването и избират правилна едно-, дву- или четирибайтова форма. 8-битовото изместване (късо) има диапазон на прескачане между +127 и -128 байта от следващата инструкция, докато 16-битовото изместване (близо) има диапазон от $\pm 32K$ байта. В 80386 и по-високи, 32-битовото изместване позволява диапазон на скока от $\pm 2GB$.

Косвено адресиране на програмата в паметта

Микропроцесорът позволява няколко форми на непряко адресиране на програмната памет за инструкциите JMP и CALL. Таблица 3-10 изброява някои индиректни инструкции за преход в програма, които могат да използват косвена регистрова адресация чрез 16-битов регистър (AX, BX, CX, DX, SP, BP, DI или SI), базова или индексна адресация ([BP], [BX], [DI] или [SI]), а също така и косвена адресация с изместване. В 80386 и по-високи може да се използва и разширен регистър за съхраняване на адреса или индиректния адрес на относителен JMP или CALL. Например, JMP EAX инструкцията прескача към адреса на прехода чрез регистър EAX. Ако 16-битов регистър съдържа адреса на прехода при JMP инструкция, скокът е близък. Например, ако регистърът BX съдържа 1000H и се изпълнява JMP BX инструкция, микропроцесорът прескача към адрес с отместване 1000H в текущия кодов сегмент. Ако регистър съдържа адреса, преходът също се счита за непряк. Например инструкцията JMP [BX] прави преход към адрес от паметта, записан в сегмента от данни, като изместването на този адрес в сегмента се съдържа в BX. На този адрес има 16-битово число, което се използва като отместване за скока.

TABLE 3-10 Examples of indirect program memory addressing

Assembly Language	Operation
JMP AX	Jumps to the current code segment location addressed by the contents of AX
JMP CX	Jumps to the current code segment location addressed by the contents of CX
JMP NEAR PTR [BX]	Jumps to the current code segment location addressed by the contents of the data segment memory location addressed by BX
JMP NEAR PTR [DI+2]	Jumps to the current code segment location addressed by the contents of the data segment memory location addressed by DI plus 2
JMP TABLE[BX]	Jumps to the current code segment location addressed by the contents of the data segment memory location addressed by TABLE plus BX
JMP ECX	Jumps to the current code segment location addressed by the contents of ECX

Режими на адресация на стековата памет

Паметта на стека се поддържа от два регистъра: указател на стека (SP или ESP) и сегментен регистър на стека (SS). Всеки път, когато дума от данни се запише в стека [Фигура 3-17 (а)], старшият байт се записват по адрес, указван с SP - 1. Младшият байт на думата се записват по адрес, указан с SP - 2. След това SP се намалява с 2, така че следващата дума от данни се съхранява в следващия свободен адрес на стековата памет. Регистърът SP/ESP винаги сочи към област от памет, разположена в сегмента на стека. Регистърът SP/ESP се добавя към SS x 10H, за да образува адреса на паметта на

стека в реалния режим. В защитен режим на работа, SS регистърът съдържа селектор, който осъществява достъп до дескриптор за основния адрес на сегмента на стека.

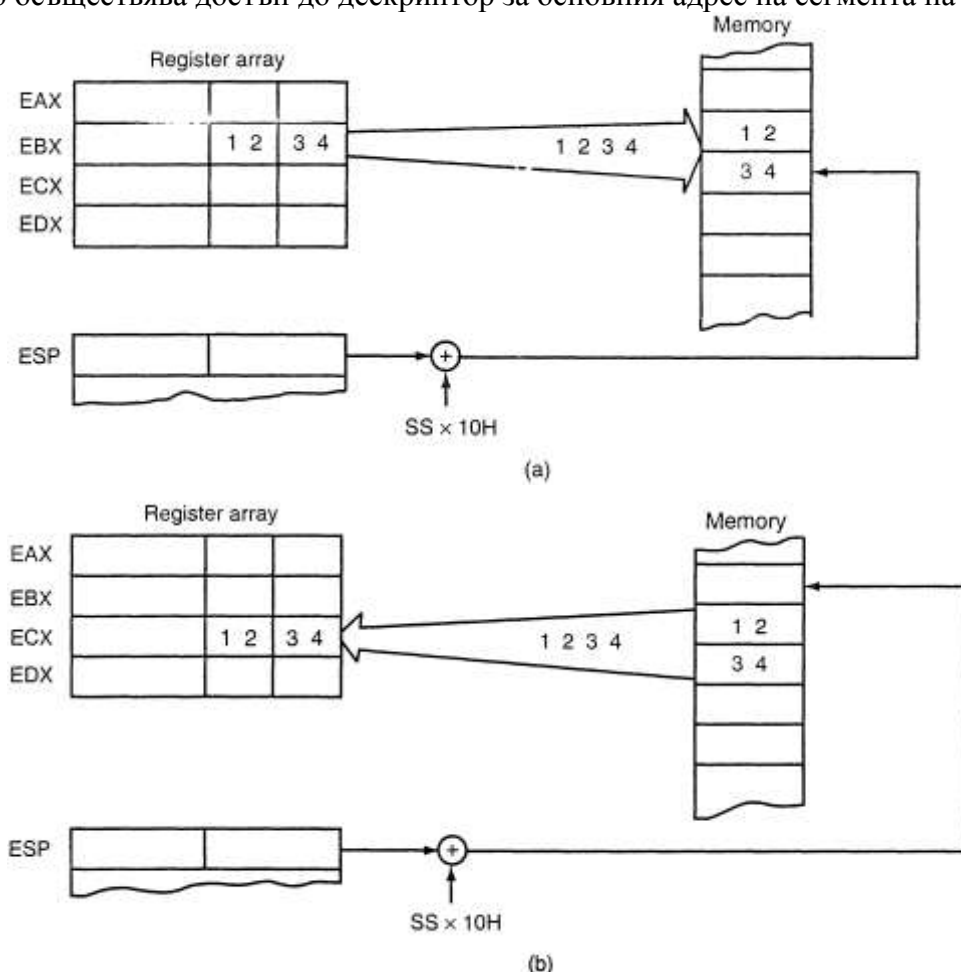


FIGURE 3-17 The PUSH and POP instructions. (a) PUSH BX places the contents of BX onto the stack, (b) POP CX removes data from the stack and places them into CX. Both instructions are shown after execution.

Методи за адресация на стековата памет

Всеки път, когато данните бъдат извлечени от стека [Фигура 3-17 (b)], младшите 8-бита се премахват от адреса, указван от SP. Старшите 8-бита се премахват от адреса, указван с SP + 1. Регистърът SP след това се увеличава с 2.

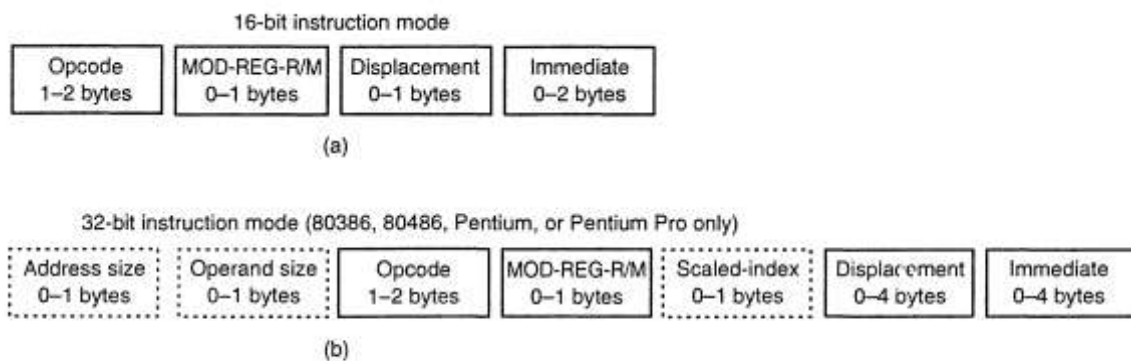
Таблица 3-11 изброява някои от PUSH и POP инструкциите, налични в микропроцесора. Инструкциите PUSH и POP винаги съхраняват или извличат думи от данни – никога байтове – в микропроцесорите 8086 до 80286. 80386 и по-горе позволяват прехвърляне на думи или двойни думи към и от стека. Данните могат да бъдат записани в стека от всеки 16-битов регистър или сегментен регистър, а в 80386 и по-високи, всеки 32-битов разширен регистър. Данните могат да бъдат извлечени от стека във всеки 16-битов регистър или всеки сегментен регистър с изключение на CS. Инструкциите PUSHA и POPA или записват, или извличат всички регистри в стека, с изключение на сегментните регистри. Тези инструкции не са налични в 8086/8088. Инструкцията за запис на непосредствен операнд е нова за 80286-Pentium Pro. В Таблица 3-11, се показва реда на регистрите, прехвърлени от инструкциите PUSHA и POPA. 80386 и по-високи позволяват в стека да бъдат записвани или извлечени разширени регистри.

TABLE 3–11 Example PUSH and POP Instructions

<i>Assembly Language</i>	<i>Operation</i>
POPF	Removes a word from the stack and places it into the flags
POPFD	Removes a doubleword from the stack and places it into the EFLAG register
PUSHF	Copies the flags onto the stack
PUSHFD	Copies the EFLAG register to the stack
PUSH AX	Copies AX to the stack
POP BX	Removes a word from the stack and places it into BX
PUSH DS	Copies DS to the stack
PUSH 1234H	Copies a 1234H to the stack
POP CS	Illegal instruction
PUSH WORD PTR [BX]	Copies a word from the data segment memory location addressed by BX onto the stack
PUSHA	Copies the word contents of AX, CX, DX, BX, SP, BP, DI, and SI onto the stack
POPA	Removes data from the stack and places it into SI, DI, BP, SP, BX, DX, CX, and AX
PUSHAD	Copies the doubleword contents of EAX, ECX, EDX, EBX, ESP, EBP, EDI, and ESI onto the stack
POPAD	Removes data from the stack and places it into ESI, EDI, EBP, ESP, EBX, EDX, ECX, and EAX
POP EAX	Removes data from the stack and places it into EAX
PUSH EDI	Copies EDI to the stack

Видове инструкции

Инструкциите за прехвърляне на данни прехвърлят данни между регистри, регистър и памет, регистър и стека, памет и стек, акумулатор и I/O, както и флаговете и стека. Прехвърлянията от памет към памет са разрешени само с инструкцията MOVS. Инструкциите за прехвърляне на данни включват MOV, PUSH, POP, XCHG, XLAT, IN, BٲH, LEA, LDS, LES, LSS, LGS, LFS, LAHF, SAHF и инструкциите за низове: LODS, STOS, MOVS, INS и OUTS. Първият байт на инструкцията съдържа кода на операцията. Операционният код определя операцията, изпълнявана от микропроцесора. Операционният код може да бъде предшестван от един или повече префикси за отмяна в някои форми на инструкции.

**FIGURE 4–1** The formats of the 8086–Pentium Pro instructions. (a) The 16-bit form and (b) the 32-bit form.

Инструкциите за 8086-80286 са инструкции за 16-б режим, които приемат формата, показана на Фигура 4-1 (а). Инструкциите за 16-битов режим са съвместими с 80386 и по-високи, ако са програмирани да работят в режим на 16-битови инструкции, но те могат да имат префикс, както е показано на Фигура 4-1 (b). 80386 и по-високи приемат, че всички инструкции са 16-битови инструкции за режим, когато машината работи в реален режим. В защитен режим старшият байт на дескриптора съдържа D-бит, който избира 16- или 32-битов режим на инструкции. Инструкциите за 32-битов режим са във формата, показана на Фигура 4-1 (b). Тези инструкции се изпълняват в режим на 16-битови инструкции чрез използване на префикси. Първите два байта от 32-битовия формат на режима на инструкциите се наричат префикси за отмяна, тъй като не винаги присъстват. Първият променя размера на адреса на операнда, използван от инструкцията, а вторият променя размера на регистъра. Ако 80386 - Pentium Pro работят като машини с 16-битов режим на инструкции (реален или защитен режим) и се използва 32-битов регистър, префиксът за размер на регистъра (66H) се добавя към предната част на инструкцията. Ако се работи в 32-битов режим на инструкции (само защитен режим) и се използва 32-битов регистър, префиксът за размер на регистъра отсъства. Ако 16-битов регистър се появи в инструкция в режим на 32-битова инструкция, присъства префиксът за размер на регистъра за избор на 16-битов регистър. Префиксът за размер на адреса (67H) се използва по подобен начин. Префиксите превключват размера на регистъра и адреса на операнда от 16-битов на 32-битов или от 32-битов на 16-битов за инструкцията с префикс.

Операционният код избира операцията (събиране, изваждане, преместване и т.н.), извършена от микропроцесора. Операционният код е дълъг един или два байта за повечето инструкции на машинния език.

FIGURE 4-2 Byte 1 of many machine language instructions, showing the position of the D- and W-bits

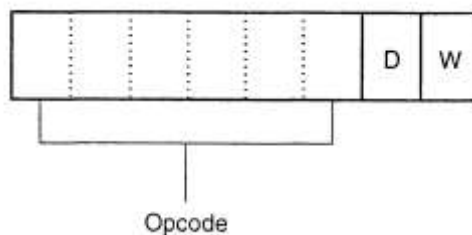
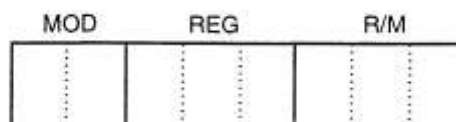


FIGURE 4-3 Byte 2 of many machine language instructions, showing the position of the MOD, REG, and R/M fields



Микропроцесорът притежава **аритметични и логически инструкции**. Аритметичните инструкции включват събиране, изваждане, умножение, деление, сравнение, отрицание, инкрементация и декрементация. Логическите инструкции включват И, ИЛИ, Изключващо-ИЛИ, НЕ, изместване, ротация и логическото сравнение (TEST). 80386 -Pentium Pro притежава инструкции XADD, SHRD, SHLD, за битови тестове и за битови сканирания. **Инструкциите за обработка на низове** работят върху масиви с 1, 2, 4 байтови елементи и са специално разработени за целта команди, с които се цели ускоряване на обработката им. Особеното в тези команди е това, че с тяхна помощ може да се изпълни определено действие върху всички елементи на низа. **Инструкциите за управление на програмата** позволяват да се промени реда на изпълнение на инструкциите. Промяната в реда обикновено се прави от инструкция за условен и безусловен преход. Към тези инструкции спадат инструкциите за скокове, извиквания, връщания, прекъсвания и инструкции за управление на процесора.